

A Minimal Mathematical Definition of a Stateless Event-Condition-Action Rule Engine

A Pure, Technology-Independent Mathematical Foundation

Rohin Gosling, 2022

***Status.** Independent technical note. This document has not been peer-reviewed. The note presents a pure mathematical definition of a stateless event-condition-action rule engine and intentionally excludes implementation technology, operational architecture, and case-study material.*

Abstract

This note gives a minimal mathematical definition of a stateless event-condition-action rule engine. An event occurrence consists of an event type and an optional payload. A payload is a finite map from parameter names to values; the empty map represents the absence of a payload. A condition is evaluated by a Boolean predicate that may inspect selected payload parameters. A rule associates one event type and one condition with one action. The engine queries the rule set and returns either the unique action identified by the current event type and payload, or no action. To keep this zero-or-one result deterministic without priorities or conflict resolution, the query is defined only when the current occurrence and rule set identify at most one distinct action. A globally admissible rule set satisfies that restriction for every occurrence. The engine is stateless because its result depends only on the fixed rule set and the current event occurrence, never on preceding occurrences. By leaving event types, conditions, payload values, and actions uninterpreted, the model provides a domain-independent reference semantics rather than a database-specific execution model. Relative to established ECA systems, the model deliberately excludes composite-event detection, temporal history, mutable condition context, transaction coupling, conflict resolution, and action execution. Its intended role is therefore not a general replacement semantics, but a small reference semantics for the occurrence-local, deterministic fragment of ECA evaluation.

Keywords: event-condition-action rules; statelessness; predicates; formal definition; domain-independent semantics; reference model

1 Aim

The aim is to formalize the following statement with as little notation as possible:

An event occurs, possibly with a payload. Conditions are evaluated, possibly using parameters from that payload. The rules are queried to identify the action associated with the event and a true condition, if such an action exists.

The complete definition will reduce to one function

$$Q : \mathcal{X} \times \mathfrak{S} \rightarrow \mathcal{A}_\perp, \quad \text{dom}(Q) = \mathfrak{D}_q, \quad (1)$$

where $\mathfrak{S} = 2^{\mathcal{E} \times \mathcal{C} \times \mathcal{A}}$ is the set of all rule sets, $\mathfrak{D}_q$ is the domain of admissible current query inputs defined below, and $\mathcal{A}_\perp = \mathcal{A} \cup \{\perp\}$ with $\perp \notin \mathcal{A}$ denoting no action. Thus every defined query returns exactly one value: either $\perp$ or one element of $\mathcal{A}$. It never returns a collection of actions. A globally admissible rule set is the special case for which the query is defined at every occurrence.

The note does not attempt to reproduce the full expressive or operational semantics of established ECA systems. It asks a narrower question: what remains of ECA evaluation when a rule set and the current event occurrence are the only inputs?

A second aim is to isolate a domain-independent interpretation of the ECA idea. The sets of events, conditions, payload values, and actions are therefore left uninterpreted. They may be instantiated for different reactive domains without changing the evaluation function. Here, *domain-independent* means independent of application-specific state and execution semantics; it does not mean that the model reproduces the full expressive power of richer ECA systems.

2 Relation to Existing ECA Work

The ECA formulation was developed in the active-database literature. In the HiPAC architecture, McCarthy and Dayal describe ECA rules together with an execution model embedded in database transactions [1]. Paton and Díaz later survey active database systems in terms of a much richer design space, including event structure and composition, event consumption, condition context, action options, rule management, and execution semantics [4].

Event processing is itself a substantial part of that design space. Snoop distinguishes primitive, temporal, and composite events and supplies operators and parameter contexts for event expressions [2]. Chakravarthy et al. give composite events semantics over a global event history and analyze contexts for selecting and consuming event occurrences [3]. Such models are intentionally history-sensitive: a composite event can depend on several occurrences and their ordering.

Later reaction-rule work extends beyond the active-database setting. Reaction RuleML, for example, provides a representation framework for rules, events, actions, complex-event processing, knowledge updates, modules, and message interchange [5]. These facilities address representation, reasoning, communication, and execution concerns that are outside the mathematical function studied here.

The present note takes a different direction from both strands. Rather than adding domain-specific or operational expressiveness, it asks whether a common occurrence-local core can be stated without committing to a database, event algebra, reasoning language, or execution environment. The resulting model is more domain-independent in its interpretation, but deliberately less expressive in its behaviour.

2.1 Deliberate Boundary

The present model retains only event matching, payload-local condition evaluation, and action selection. It excludes:

- event histories, temporal windows, event algebras, and composite-event consumption;
- condition queries over mutable external or database state;
- event-condition and condition-action coupling modes, transactions, and rollback;
- priorities, conflict resolution, rule cascades, and recursive event generation; and
- the operational meaning, ordering, success, or side effects of executing the returned action.

The domain independence of the model follows from these limited commitments. Event types and actions are abstract elements, payloads carry only named values, and conditions are represented only through their Boolean predicate. The model fixes no application-specific interpretation of these elements.

Because the model defines neither priority nor conflict resolution, a current occurrence and rule set that identify two distinct actions form an input outside the query's domain. This occurrence-local restriction is stated formally below. Several rules may still match when they all identify the same action, and a globally admissible rule set satisfies the restriction at every occurrence.

This exclusion is a semantic boundary, not a claim that the omitted features are unimportant. A richer ECA system may contain an occurrence-local fragment whose optional returned action can be compared with $Q((e, p), \mathcal{R})$ whenever that query is defined. If history, mutable state, event consumption, or earlier actions can change which action is returned for the same current occurrence and rule set, that behaviour lies outside this fragment.

2.2 Potential Value of the Restricted Model

The restriction may be useful in five roles.

1. *Reference semantics.* It provides a small mathematical oracle against which an occurrence-local evaluator can be checked.
2. *Conformance boundary.* It separates action selection from scheduling, transactions, side effects, and delivery, so disagreement can be localized to one semantic layer.
3. *Baseline for comparison.* Richer ECA formalisms can state precisely which additional arguments or operators make their behaviour history-dependent or state-dependent.
4. *Extension kernel.* Composite-event detection, state access, priorities, and action execution can be specified as explicit extensions rather than being implicit in the meaning of a rule.
5. *Cross-domain reference model.* Different ECA applications may instantiate the abstract sets differently while retaining the same single-action query semantics, allowing their occurrence-local behaviour to be described and compared through a common mathematical interface.

These uses motivate the isolated definition, but do not by themselves establish novelty over the broader ECA literature. The contribution claimed here is the explicit restriction and its mathematical boundary.

3 Events and Optional Payloads

Let K be a set of parameter names and let V be a set of parameter values.

Definition 3.1 (Payload). *A payload is a finite partial function*

$$p : K \rightharpoonup V. \quad (2)$$

The set of all payloads is $\mathcal{P}$. The empty function $\emptyset \in \mathcal{P}$ represents no payload.

If $k \in \text{dom}(p)$, then the parameter k is present and has value $p(k)$. If $k \notin \text{dom}(p)$, then the parameter is absent.

Definition 3.2 (Event occurrence). *Let $\mathcal{E}$ be a set of event types. An event occurrence is a pair*

$$x = (e, p) \in \mathcal{X} = \mathcal{E} \times \mathcal{P}. \quad (3)$$

The occurrence $(e, \emptyset)$ is an event with no payload.

This definition treats an event type e and an occurrence (e, p) as different objects. The event type identifies what occurred; the payload supplies the parameters associated with that particular occurrence. The set $\mathcal{E}$ is the universe of possible event types, not merely the event types named by one finite rule set. An occurrence whose event type appears in no rule is therefore still in $\mathcal{X}$ and has an empty candidate-action set.

4 Conditions and Their Predicate

Let C be a set of conditions. Every condition $c \in C$ has a finite dependency set

$$K_c \subseteq K, \quad (4)$$

containing the payload parameters that the condition may use.

Definition 4.1 (Condition predicate). *The condition predicate is a total function*

$$q : C \times \mathcal{P} \longrightarrow \{0, 1\}, \quad (5)$$

where $q(c, p) = 1$ means that condition c is true for payload p , and $q(c, p) = 0$ means that it is false.

The dependency set limits which parameters may affect a condition. If two payloads agree on every parameter in K_c , including which of those parameters are absent, then

$$q(c, p) = q(c, p'). \quad (6)$$

The minimal convention for a missing required parameter is

$$K_c \not\subseteq \text{dom}(p) \implies q(c, p) = 0. \quad (7)$$

Two simple cases follow immediately.

- If $K_c = \emptyset$, then c does not use the payload.
- A distinguished condition $\top \in C$ may be defined by $K_\top = \emptyset$ and $q(\top, p) = 1$ for every p . It represents an unconditional rule.

Thus the same predicate covers conditions that use payload parameters and conditions that do not.

5 Actions and Rules

Let $\mathcal{A}$ be a set of actions.

Definition 5.1 (Rule). *A rule is a triple*

$$r = (e, c, a) \in \mathcal{E} \times C \times \mathcal{A}. \quad (8)$$

It states that action a is a candidate when an event of type e occurs and condition c is true.

Let

$$\mathcal{R} \subseteq \mathcal{E} \times C \times \mathcal{A} \quad (9)$$

be the set of rules.

The Boolean query predicate tests whether an action is identified by at least one matching rule:

$$M : \mathcal{X} \times 2^{\mathcal{E} \times C \times \mathcal{A}} \times \mathcal{A} \longrightarrow \{0, 1\}, \quad (10)$$

$$M((e, p), R, a) = 1 \iff \exists c \in C : (e, c, a) \in R \wedge q(c, p) = 1. \quad (11)$$

For a rule set R , its candidate-action set at an occurrence is therefore

$$C_R(e, p) = \{a \in \mathcal{A} \mid M((e, p), R, a) = 1\}. \quad (12)$$

Let

$$\mathfrak{S} = 2^{\mathcal{E} \times C \times \mathcal{A}} \quad (13)$$

be the set of all rule sets.

Definition 5.2 (Admissible query input). *An occurrence and rule set form an admissible query input when their candidate-action set contains at most one action. Let*

$$\mathfrak{D}_q = \{((e, p), R) \in \mathcal{X} \times \mathfrak{S} \mid |C_R(e, p)| \leq 1\} \quad (14)$$

be the domain of admissible query inputs for the fixed condition predicate q .

Definition 5.3 (Globally admissible rule set). *A rule set is globally admissible when it forms an admissible query input with every occurrence. Let*

$$\mathfrak{R}_q = \{R \in \mathfrak{S} \mid \forall x \in \mathcal{X} : (x, R) \in \mathfrak{D}_q\} \quad (15)$$

be the family of globally admissible rule sets for q .

Several rules may match the same occurrence when they all identify the same action. If simultaneously matching rules identify distinct actions, that occurrence and rule set do not form an admissible query input; choosing between those actions would require a priority or conflict-resolution policy that this minimal model does not define. The same rule set may still form admissible inputs with other occurrences, but it is not globally admissible.

6 The Rule Engine

Definition 6.1 (Stateless ECA rule-set query). *The rule engine is the partial option-valued query*

$$Q : \mathcal{X} \times \mathfrak{S} \rightarrow \mathcal{A}_\perp, \quad \text{dom}(Q) = \mathfrak{D}_q. \quad (16)$$

For every admissible query input $((e, p), R) \in \mathfrak{D}_q$, it is defined by

$$Q((e, p), R) = \begin{cases} a, & C_R(e, p) = \{a\}, \\ \perp, & C_R(e, p) = \emptyset. \end{cases} \quad (17)$$

For a fixed rule set $\mathcal{R} \in \mathfrak{S}$, write

$$Q_{\mathcal{R}} : \mathcal{X} \rightarrow \mathcal{A}_\perp, \quad \text{dom}(Q_{\mathcal{R}}) = \{x \in \mathcal{X} \mid (x, \mathcal{R}) \in \mathfrak{D}_q\}, \quad Q_{\mathcal{R}}(e, p) = Q((e, p), \mathcal{R}). \quad (18)$$

If $\mathcal{R} \in \mathfrak{R}_q$, then $\text{dom}(Q_{\mathcal{R}}) = \mathcal{X}$ and this fixed-rule query is total. If $|C_{\mathcal{R}}(e, p)| > 1$, then Q is undefined at $((e, p), \mathcal{R})$: the result is not $\perp$, and the model does not choose an action.

Admissibility of the current query input guarantees that the two cases in Equation (17) are exhaustive and that the action a in the first case is unique. This is the formal counterpart of the target expression

$$\text{action} = Q(\text{event}(\text{payload}), \text{rules}). \quad (19)$$

Here, $\text{event}(\text{payload})$ denotes the occurrence (e, p) . The Boolean predicate M performs the candidate test; the action-valued operator Q returns the predicate's unique satisfying action, or $\perp$ when there is none, whenever the query input belongs to $\mathfrak{D}_q$.

Equation (17) captures the complete minimal model:

1. $\mathcal{E}$ is the set of event types;
2. $\mathcal{P}$ is the set of optional payloads;
3. $\mathcal{C}$ is the set of conditions;
4. q evaluates a condition against a payload;
5. $\mathcal{A}$ is the set of actions;
6. $\mathcal{R}$ relates events and conditions to actions;
7. M tests whether a candidate action is identified by the occurrence and rule set; and
8. Q maps an admissible current occurrence and rule-set input to one action or $\perp$.

If no rule identifies an action, the result is $\perp$. If one or more matching rules all identify the same action, that action is returned. A defined query never returns more than one action. Distinct candidate actions make the input inadmissible rather than producing a second kind of no-action result.

7 Statelessness

Let X^* be the set of finite sequences of event occurrences. Such a sequence is called a history. A possibly history-dependent behaviour has the form

$$G : X^* \times X \longrightarrow \mathcal{A}_\perp. \quad (20)$$

Definition 7.1 (Stateless evaluation). *A behaviour G is stateless when its result for the current occurrence does not depend on a history: for all histories $h_1, h_2 \in X^*$ and every $x \in X$,*

$$G(h_1, x) = G(h_2, x). \quad (21)$$

For a fixed globally admissible $\mathcal{R} \in \mathfrak{R}_q$, the rule engine induces the behaviour

$$G_{\mathcal{R}}(h, (e, p)) = Q_{\mathcal{R}}(e, p) = Q((e, p), \mathcal{R}). \quad (22)$$

Equation (22) does not use h , so $G_{\mathcal{R}}$ satisfies (21).

Proposition 7.2 (Factorization of stateless behaviour). *A behaviour G is stateless if and only if there is a unique function*

$$f : X \longrightarrow \mathcal{A}_\perp \quad (23)$$

such that $G(h, x) = f(x)$ for every history h and occurrence x .

Proof. If G is stateless, let ϵ be the empty history and define $f(x) = G(\epsilon, x)$. Then $G(h, x) = G(\epsilon, x) = f(x)$ for every h by (21). Any other such function must equal $G(\epsilon, x)$ at every x , so f is unique. Conversely, if $G(h, x) = f(x)$, then $G(h_1, x) = f(x) = G(h_2, x)$ for all histories h_1 and h_2 . $\square$

For the rule engine, the unique factor is $f(e, p) = Q_{\mathcal{R}}(e, p)$. This factorization states the exact abstraction made by the model: history is not compressed or hidden; it is mathematically irrelevant to the optional returned action.

Proposition 7.3 (Determinism). *For a fixed globally admissible $\mathcal{R} \in \mathfrak{R}_q$, condition predicate q , and occurrence (e, p) , the result $Q_{\mathcal{R}}(e, p) \in \mathcal{A}_\perp$ is unique.*

Proof. Every condition value $q(c, p)$ is uniquely either 0 or 1, so the candidate-action set $C_{\mathcal{R}}(e, p)$ is uniquely determined. Admissibility makes that set either empty or a singleton. Equation (17) therefore returns exactly one uniquely determined value: $\perp$ in the empty case, or the singleton's action in the other case. $\square$

Corollary 7.4 (Replay). *Querying the same occurrence against the same globally admissible rule set always returns the same optional action.*

8 Immediate Consequences

For an event type e and fixed globally admissible rule set $\mathcal{R} \in \mathfrak{R}_q$, let

$$K_{e,\mathcal{R}} = \bigcup \{K_c \mid (e, c, a) \in \mathcal{R}\} \quad (24)$$

be the set of all payload parameters used by rules for e .

Proposition 8.1 (Payload locality). *If payloads p and p' agree on every parameter in $K_{e,\mathcal{R}}$, including absence, then*

$$Q_{\mathcal{R}}(e, p) = Q_{\mathcal{R}}(e, p'). \quad (25)$$

Proof. Every condition attached to e has the same value under p and p' by (6). The query predicate therefore produces the same candidate-action set, and Equation (17) returns the same optional action. $\square$

Corollary 8.2 (Payload independence). *If $K_{e,\mathcal{R}} = \emptyset$, then the optional action returned for event type e is independent of its payload.*

Proposition 8.3 (Admissible extension stability). *Let $R, R' \in \mathfrak{R}_q$ with $R \subseteq R'$. For every occurrence (e, p) and action $a \in \mathcal{A}$,*

$$Q((e, p), R) = a \implies Q((e, p), R') = a. \quad (26)$$

Proof. If $Q((e, p), R) = a$, then $a \in C_R(e, p)$. Since $R \subseteq R'$, the same matching rule belongs to R' , so $a \in C_{R'}(e, p)$. Admissibility of R' permits no distinct candidate action, and therefore $C_{R'}(e, p) = \{a\}$. Equation (17) returns a . $\square$

An admissible extension may change $\perp$ into an action by adding the first matching rule. It cannot replace an already returned action with a distinct action; an extension that attempted to do so would not be globally admissible. Global admissibility makes both query inputs in Equation (26) well defined.

Finally, any behaviour that produces different optional actions for the same current occurrence and rule set after two different histories cannot be represented by (17). Such a behaviour is history-dependent and lies outside the stateless model.

9 Conclusion

The minimal stateless event-condition-action rule engine is completely described by

$$\begin{aligned}
 \mathfrak{S} &= 2^{\mathcal{E} \times \mathcal{C} \times \mathcal{A}}, & q : \mathcal{C} \times \mathcal{P} &\rightarrow \{0, 1\}, \\
 C_{\mathcal{R}}(e, p) &= \{a \in \mathcal{A} \mid \exists c : (e, c, a) \in \mathcal{R} \wedge q(c, p) = 1\}, \\
 \text{dom}(Q) &= \{((e, p), \mathcal{R}) \in \mathcal{X} \times \mathfrak{S} \mid |C_{\mathcal{R}}(e, p)| \leq 1\}, \\
 Q((e, p), \mathcal{R}) &= \begin{cases} a, & C_{\mathcal{R}}(e, p) = \{a\}, \\ \perp, & C_{\mathcal{R}}(e, p) = \emptyset. \end{cases}
 \end{aligned} \tag{27}$$

The payload p may be empty. The predicate q may ignore the payload or inspect selected parameters from it. A rule relates an event and a condition to an action. The query returns the unique action named by the matching rules, or $\perp$ when no rule matches. If the matching rules identify distinct actions, that occurrence and rule set lie outside $\text{dom}(Q)$; the query does not silently choose between them. A globally admissible rule set has no such occurrence and makes its fixed-rule query total. For a fixed rule set, the current occurrence is the only varying argument, so the engine is stateless throughout its domain. In relation to richer ECA systems, this function should be read as a reference semantics for an optional action query after a single event occurrence has been identified, not as a semantics for event-history recognition, transactions, conflict resolution, or action execution. Because the event, condition, payload, and action domains remain abstract, the same kernel can be instantiated across reactive domains without introducing database, platform, or execution assumptions.

References

- [1] D. R. McCarthy and U. Dayal, “The architecture of an active database management system,” in *Proceedings of the 1989 ACM SIGMOD International Conference on Management of Data*, pp. 215–224, 1989. doi:10.1145/66926.66946.
- [2] S. Chakravarthy and D. Mishra, “Snoop: An expressive event specification language for active databases,” *Data & Knowledge Engineering*, vol. 14, no. 1, pp. 1–26, 1994. doi:10.1016/0169-023X(94)90006-X.
- [3] S. Chakravarthy, V. Krishnaprasad, E. Anwar, and S.-K. Kim, “Composite events for active databases: Semantics, contexts and detection,” in *Proceedings of the 20th International Conference on Very Large Data Bases*, pp. 606–617, 1994. vldb.org/conf/1994/P606.PDF.
- [4] N. W. Paton and O. Díaz, “Active database systems,” *ACM Computing Surveys*, vol. 31, no. 1, pp. 63–103, 1999. doi:10.1145/311531.311623.
- [5] A. Paschke, “Reaction RuleML 1.0 for rules, events and actions in semantic complex event processing,” in *Rules on the Web: From Theory to Applications*, Lecture Notes in Computer Science, vol. 8620, pp. 1–21, Springer, 2014. doi:10.1007/978-3-319-09870-8_1.